

Передача данных в Веб

Александр Менщиков,
к.т.н., доцент ИТМО

Содержание лекции

1. Введение в протокол HTTP
2. Cookie
3. Передача файлов
4. Инструменты передачи данных
5. Polling & Ajax
6. Git
7. VPN сеть на LMS2.0

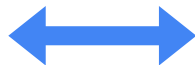
Введение в протокол HTTP

Архитектура веб приложения

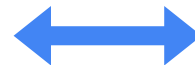
- Веб браузер
- Веб сервер
- Веб приложение



Web browser



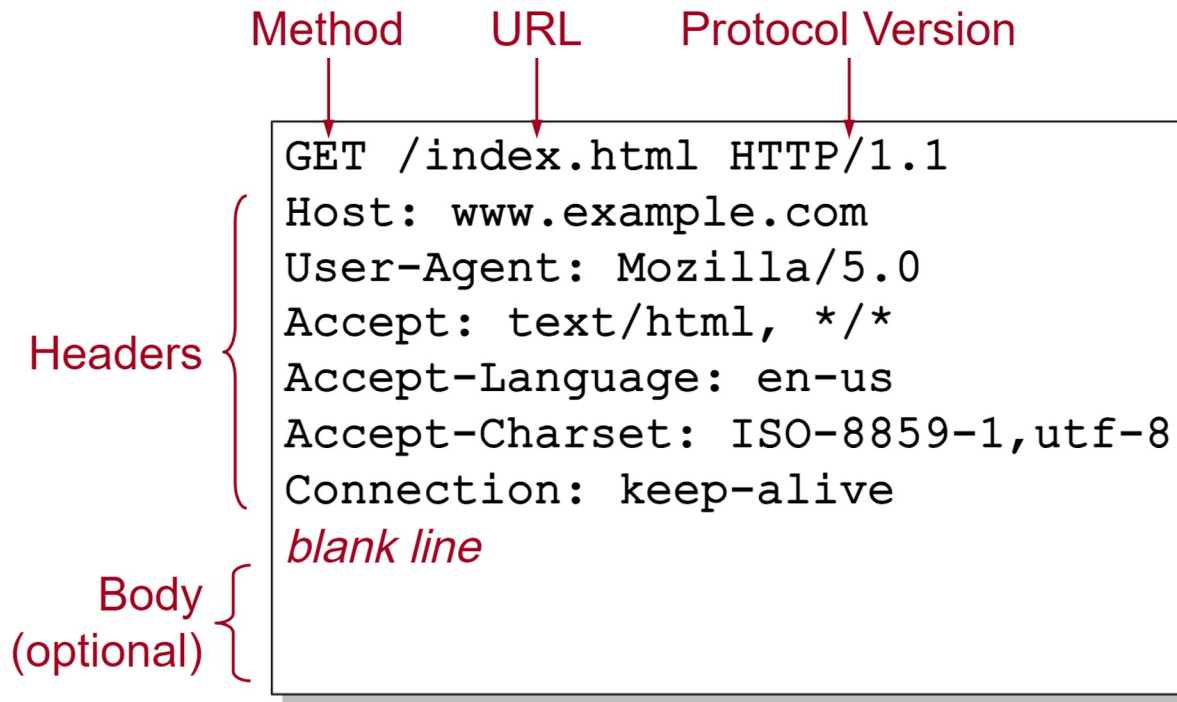
Web server



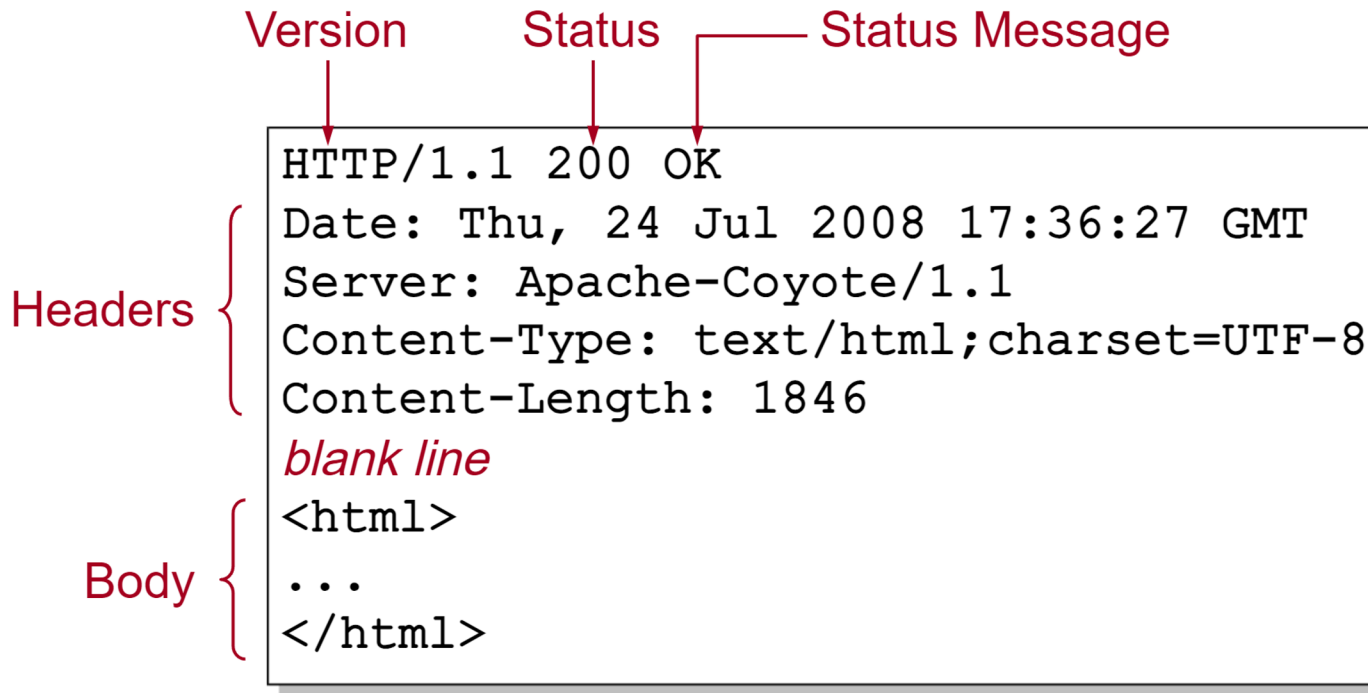
Web application

HTTP запрос

HTTP Request



HTTP Response



HTTP методы

HTTP определяет множество методов запроса, которые указывают, какое желаемое действие выполнится для данного ресурса.

GET, HEAD, POST, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH

...

```
curl -X COPY http://localhost:5001
```

```
127.0.0.1 - - [15/Feb/2024 14:57:33] "COPY / HTTP/1.1" 405 -
```

9.9. MOVE Method	56
9.9.1. MOVE for Properties	57
9.9.2. MOVE for Collections	57
9.9.3. MOVE and the Overwrite Header	58
9.9.4. Status Codes	59
9.9.5. Example - MOVE of a Non-Collection	60
9.9.6. Example - MOVE of a Collection	60
9.10. LOCK Method	61
9.10.1. Creating a Lock on an Existing Resource	61
9.10.2. Refreshing Locks	62
9.10.3. Depth and Locking	62
9.10.4. Locking Unmapped URLs	63
9.10.5. Lock Compatibility Table	63
9.10.6. LOCK Responses	63
9.10.7. Example - Simple Lock Request	64
9.10.8. Example - Refreshing a Write Lock	65
9.10.9. Example - Multi-Resource Lock Request	66
9.11. UNLOCK Method	68
9.11.1. Status Codes	68

<https://datatracker.ietf.org/doc/html/rfc4918#section-9.8>

Статус коды

- 1xx: Informational
 - 2xx: Success
 - 3xx: Redirection
 - 4xx: Client Error
 - 5xx: Server Error
- 200 OK
 - 301 Moved Permanently
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 500 Internal Server Error
 - 502 Bad Gateway
 - 504 Gateway Timeout.

HTTP заголовки

Content-Type	Тип данных в теле запроса
Content-Length	Длина запроса в байтах
Host	Запрашиваемый виртуальный хост
Accept-Language	Принимаемый язык
Accept-Encoding	Принимаемые кодировки
Authorization	Поле для различных типов авторизации

Пути передачи информации

- 1. В строке запроса
 - 2. В параметрах GET запроса
 - 3. В HTTP заголовках
 - 4. В Cookie
 - 5. В теле HTTP запроса
- Request-Line*
- Headers*
- Message-body*

```
Request      = Request-Line           ; Section 5.1
               *(( general-header      ; Section 4.5
                  | request-header     ; Section 5.3
                  | entity-header ) CRLF) ; Section 7.1
               CRLF
               [ message-body ]       ; Section 4.3
```

Сокращения в RFC

OCTET	= <any 8-bit sequence of data>
CHAR	= <any US-ASCII character (octets 0 - 127)>
UPALPHA	= <any US-ASCII uppercase letter "A".."Z">
LOALPHA	= <any US-ASCII lowercase letter "a".."z">
ALPHA	= UPALPHA LOALPHA
DIGIT	= <any US-ASCII digit "0".."9">
CTL	= <any US-ASCII control character (octets 0 - 31) and DEL (127)>
CR	= <US-ASCII CR, carriage return (13)>
LF	= <US-ASCII LF, linefeed (10)>
SP	= <US-ASCII SP, space (32)>
HT	= <US-ASCII HT, horizontal-tab (9)>
<">	= <US-ASCII double-quote mark (34)>

ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

1. В строке запроса

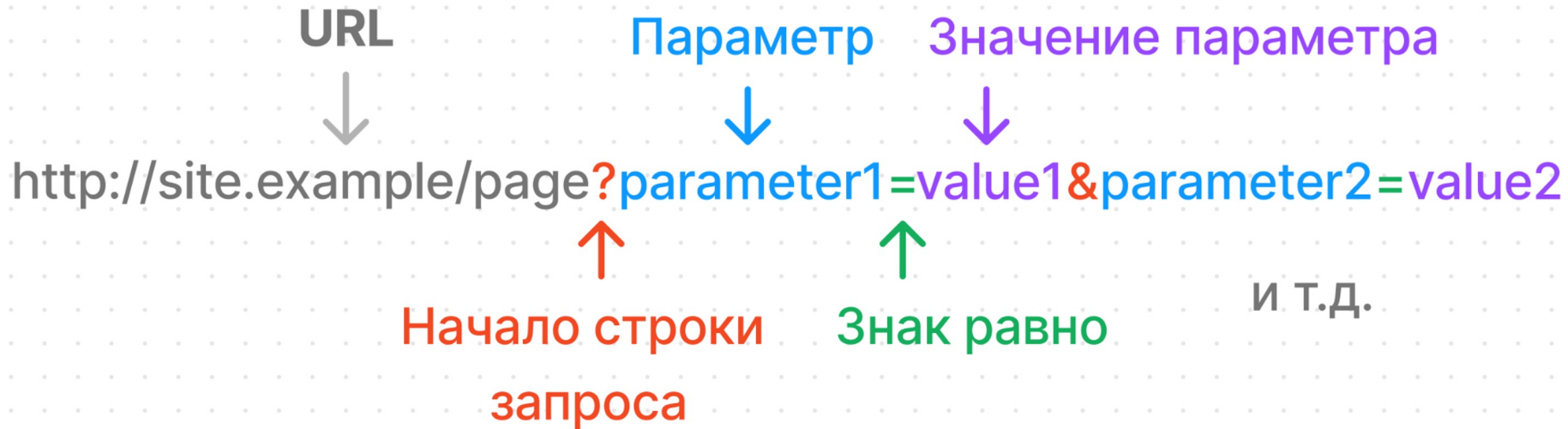
Строка запроса начинается с **маркера метода**, затем следует **Request-URI**, **версия протокола** и заканчивается **CRLF**. Элементы разделяются символами SP. Не допускаются CR или LF, за исключением заключительной последовательности CRLF.

A terminal window with a dark background and a light gray border. It features three colored window control buttons (red, yellow, green) in the top-left corner. The text inside the terminal is a curl command: `curl http://localhost:5001/information/message/data`. The word 'curl' is in blue, and the rest of the URL is in red.

SP = <US-ASCII SP, space (32)>
Request-URI = "*" | absoluteURI | abs_path | authority

<https://www.w3.org/Protocols/rfc2616/rfc2616-sec5.html>

2. В параметрах GET запроса



```
curl http://localhost:5001/register?name=123&password=secret
```

3. В HTTP заголовках

GET /flag HTTP/1.1

User-Agent: 'action'



```
curl -H "User-Agent: mycustomagent" http://localhost:5001/
```

4. B Cookie

```
GET / HTTP/1.1  
Host: localhost:5001  
User-Agent: curl/8.4.0  
Accept: */*  
Cookie: username=admin
```



```
curl --cookie "username=admin" http://localhost:5001
```

5. В теле HTTP запроса

POST / HTTP/1.1

Host: localhost:5001

User-Agent: curl/8.4.0

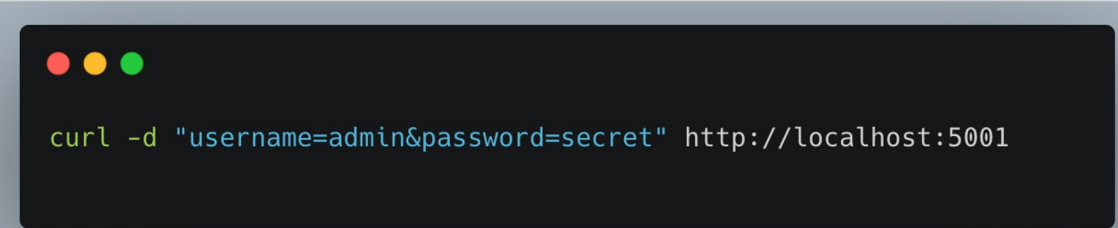
Accept: */*

Content-Length: **30**

Content-Type: application/x-www-form-urlencoded

username=admin&password=secret

30

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a single line of text: `curl -d "username=admin&password=secret" http://localhost:5001`

```
curl -d "username=admin&password=secret" http://localhost:5001
```

URL Encoding

- Данные в HTTP передаются в текстовом виде
- Для передачи бинарных данных и зарезервированных символов используется URL Encoding, примеры:

Символ	Замена
!	%21
#	%23
?	%3F

<http://site.com/login.php?username=admin&password=%21%40%23p4%24%24w0rd>
[https://gchq.github.io/CyberChef/#recipe=URL_Encode\(false\)&input=VVJMIEVuY29kaW5n](https://gchq.github.io/CyberChef/#recipe=URL_Encode(false)&input=VVJMIEVuY29kaW5n)

Cookie

HTTP. Cookie

- HTTP не хранит состояние (stateless)

- Для хранения состояния на сервере используются cookie, заголовок:

Cookie: **yummy_cookie=choco**

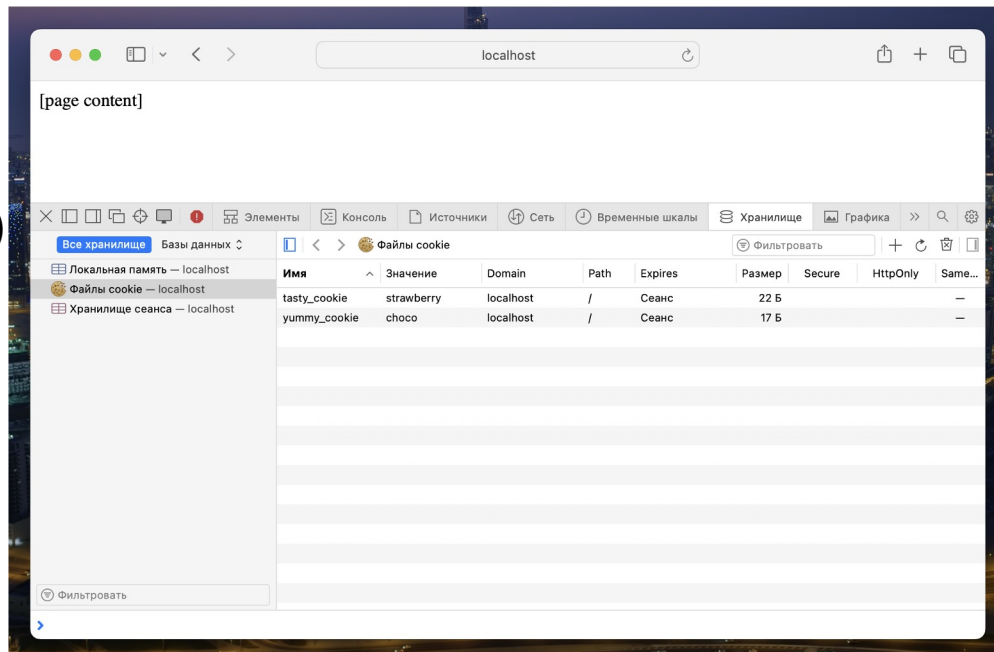
- Установка cookie:

Set-Cookie:

yummy_cookie=choco;

tasty_cookie=strawberry;

<param>=<value>



HTTP/1.0 200 OK

Content-type: text/html

Set-Cookie: yummy_cookie=choco

Set-Cookie: tasty_cookie=strawberry

[page content]

Заголовки HTTP Cookie

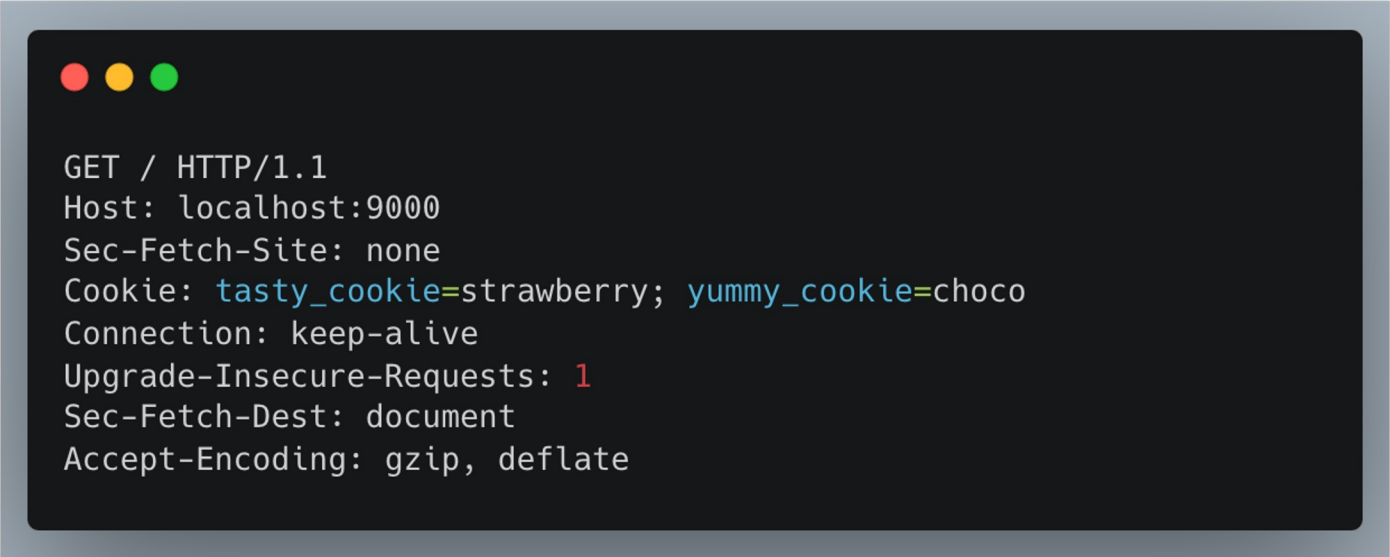
Заголовок	Пример значения	Назначение
Expires	Wed, 21 Oct 2015 07:28:00 GMT	Максимальное время жизни cookie
HttpOnly		Cookie недоступен из JavaScript
Secure		Cookie используется только в HTTPS
Path	/	Путь, для которого выставлен cookie

Браузер отправляет Cookie при каждом запросе

GET /sample_page.html HTTP/1.1

Host: www.example.org

Cookie: yummy_cookie=choco; tasty_cookie=strawberry



```
GET / HTTP/1.1
Host: localhost:9000
Sec-Fetch-Site: none
Cookie: tasty_cookie=strawberry; yummy_cookie=choco
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate
```

Время жизни Cookie

Сессионные cookie – такие cookie удаляются при закрытии клиента, то есть существуют только на протяжении текущего сеанса, поскольку атрибуты **Expires** или **Max-Age** для него не задаются.

Постоянные cookie удаляются не с закрытием клиента, а при наступлении определенной даты (атрибут **Expires**) или после определенного интервала времени (атрибут **Max-Age**).

```
Set-Cookie: id=a3fWa; Expires=Wed, 21 Feb 2024 07:30:00 GMT;
```

Доступ к Cookie

- Browser DevTools
- JavaScript
- Network

```
document.cookie = "yummy_cookie=choco";  
document.cookie = "tasty_cookie=strawberry";  
console.log(document.cookie);  
// выведет "yummy_cookie=choco; tasty_cookie=strawberry"
```

Storage

- ▶  Local storage
- ▶  Session storage
-  IndexedDB
-  Web SQL
- ▼  Cookies
 -  <https://www.w3.org>
 -  Private state tokens
 -  Interest groups
 - ▶  Shared storage
 -  Cache storage

Кавычки и whitespace в Cookie

<https://www.ietf.org/rfc/rfc2109.txt>

TOKEN – informally, a sequence of non-special, non-white space characters

av-pairs	=	av-pair *("; av-pair)
av-pair	=	attr ["=" value] ; optional value
attr	=	token
value	=	word
word	=	token quoted-string

<https://developer.mozilla.org/ru/docs/Web/HTTP/Headers/Set-Cookie#cookie-namecookie-value>

The syntax for the Set-Cookie response header is

set-cookie	=	"Set-Cookie:" cookies
cookies	=	1#cookie
cookie	=	NAME "=" VALUE *("; cookie-av)
NAME	=	attr
VALUE	=	value
cookie-av	=	"Comment" "=" value
		"Domain" "=" value
		"Max-Age" "=" value
		"Path" "=" value
		"Secure"
		"Version" "=" 1*DIGIT

Передача файлов

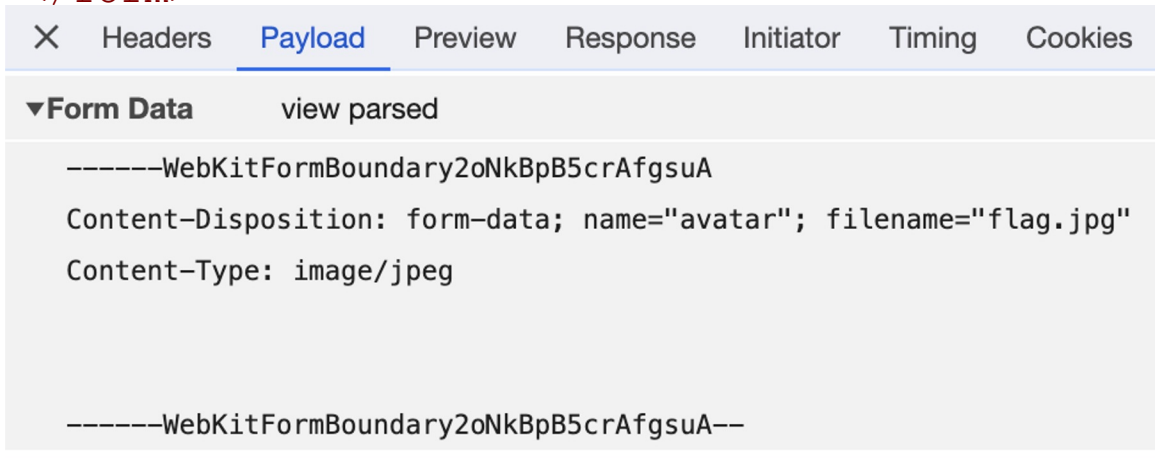
Форма загрузки файла

```
<form action="/" method="POST" enctype="multipart/form-data">  
  Upload avatar: <br>  
  <input type="file" name="avatar"><br><br>  
  <input type="submit" value="Submit">  
</form>
```

Upload avatar:

Выберите файл Файл не выбран

Submit



HTTP запрос с файлом

```
POST / HTTP/1.1
Host: localhost:5001
Connection: keep-alive
Content-Length: 19182
Cache-Control: max-age=0
sec-ch-ua: "Not A(Brand";v="99", "Google Chrome";v="121", "Chromium";v="121"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "macOS"
Upgrade-Insecure-Requests: 1
Origin: http://localhost:5001
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary6rZ8NTC7RDVdT6U1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost:5001/
Accept-Encoding: gzip, deflate, br
Accept-Language: ru,ru-RU;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: somecookie="I am cookie"

----WebKitFormBoundary6rZ8NTC7RDVdT6U1
Content-Disposition: form-data; name="avatar"; filename="flag.jpg"
Content-Type: image/jpeg

.....JFIF.....x.x.....C.....
...

.....
.....C.....l....."
```


multipart/form-data

<https://www.w3.org/TR/html401/interact/forms.html#h-17.13.4.2>

-----WebKitFormBoundary6rZ8NTC7RDVdT6U1

boundary

Content-Disposition: form-data; name="avatar"; filename="flag.jpg"

Content-Type: image/jpeg

-----WebKitFormBoundary6rZ8NTC7RDVdT6U1--

- Boundary – разделитель (не должен встречаться нигде в данных)
- [Content-Disposition](#)
- Control name

MIME type vs Magic Number

MIME предназначен для браузеров и веб-серверов

https://developer.mozilla.org/ru/docs/Web/HTTP/Basics_of_HTTP/MIME_types

Magic Number – физическая сигнатура файла, которую можно использовать для определения типа файла

https://en.wikipedia.org/wiki/List_of_file_signatures

Отправляем форму

POST / HTTP/1.1

Host: example.org

Content-Type: multipart/form-data; boundary="**boundary**"

Content-Length: 187

--boundary

Content-Disposition: form-data; name="field1"; filename="flag.png"

value1

--boundary

Content-Disposition: form-data; name="field2"; filename="example.txt"

value2

--boundary--

Инструменты передачи данных

cURL

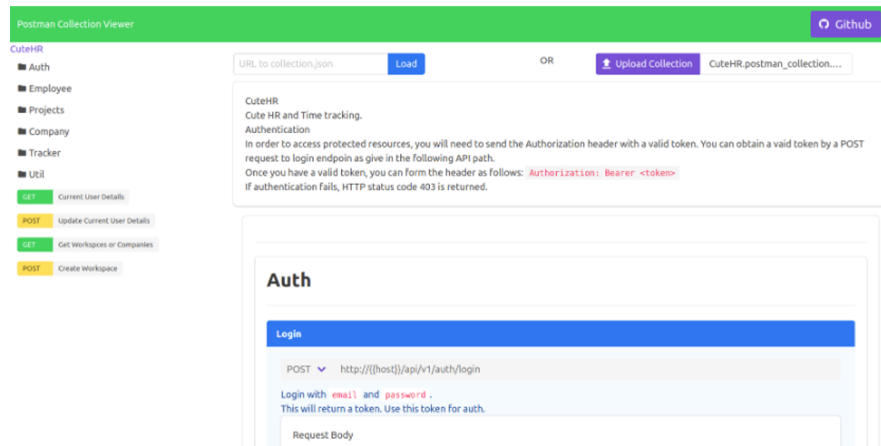
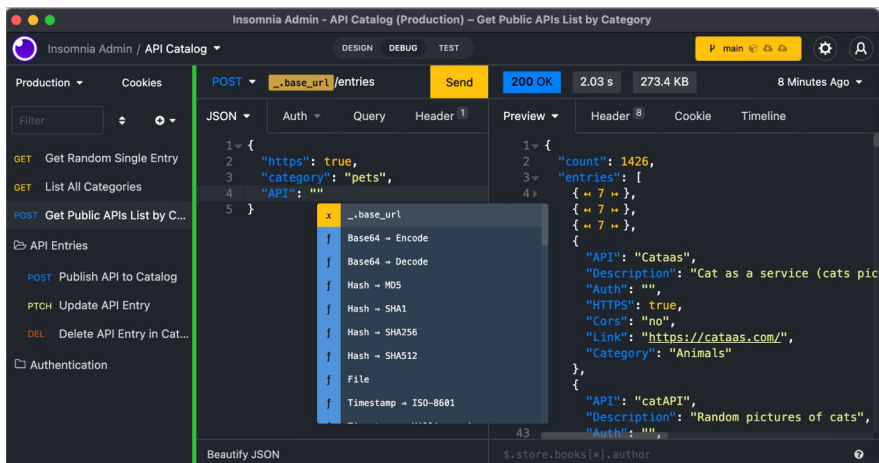
Утилита Linux для работы с HTTP

<code>curl http://ya.ru</code>	GET-запрос
<code>curl -H "User-Agent: browser" http://site.com</code>	Передача заголовков
<code>curl --data "name=user&pass=pass" http://site.com</code>	POST-запрос

GUI

Приложения для тестирования API и выполнения HTTP-запросов:

- Burp Suite
- Insomnia
- Postman



Postman

POST

http://localhost:5001

Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	KEY	VALUE	CONTENT TYPE	DESCRIPTION	...	Bulk Edit
☒	flag1	File Select Files	Auto			
	Key	Text File Value	Auto	Description		

<https://www.postman.com/postman/workspace/postman-answers/example/13455110-ff0902d4-ebe1-48fe-b805-dd8d346fd2ef>

Python requests

Библиотека Python для работы с HTTP

<https://requests.readthedocs.io/en/latest/>

```
import requests
r = requests.get("127.0.0.1")    # GET-запрос
print(r.status_code)            #
Status code
print(r.text)
# Тело ответа
import requests
r = requests.post("127.0.0.1", data={"param":
"value"})
# POST-запрос
print(r.status_code)            #
Status code
print(r.text)
```


Python requests send file

```
import requests
```

```
r = requests.post("http://localhost:5001", files={
    "Form": open("templates/form.html", "rb"),
    "Form2": ('hahaha.txt', open("templates/form.html", "rb"), 'text/x-spam')
})
print(r.text)
```

```
12 python3 05_send_file.py
{
  "files": [
    "File Form with Filename=form.html and Type=None",
    "File Form2 with Filename=hahaha.txt and Type=text/x-spam"
  ]
}
```

Powershell

Командный интерпретатор Windows

Имеет встроенные функции для работы с HTTP

```
Invoke-WebRequest -URI https://site.com -Headers @{'Accept' =  
'application/json'; 'X-My-Header' = 'Hello World'}
```

```
$postParams = @{username='me';data='qwerty'}  
Invoke-WebRequest -Uri http://site.com -Method POST -Body $postParams
```

JS (fetch)

JavaScript может отправлять сетевые запросы на сервер и подгружать новую информацию по мере необходимости.

Для сетевых запросов из JavaScript есть широко известный термин «AJAX» (аббревиатура от Asynchronous JavaScript And XML). XML мы использовать не обязаны, просто термин старый, поэтому в нём есть это слово.

```
let response = await fetch(url);

if (response.ok) { // если HTTP-статус в диапазоне 200-299
  // получаем тело ответа (см. про этот метод ниже)
  let json = await response.json();
} else {
  alert("Ошибка HTTP: " + response.status);
}
```

HTTP через сокет

Netcat (nc)

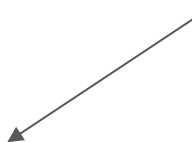
Netcat является одним из самых универсальных сетевых инструментов для системных администраторов - его называют швейцарским армейским ножом сетевых технологий.

```
nc [options] host port
```

^D

Символ **^D** (также известный как **\04** или **0x04**, **END OF TRANSMISSION** в Unicode) является значением по умолчанию для параметра специального управляющего символа **EOF** драйвера терминала или псевдотерминала.

```
n, err := conn.Read(tmp)
    if err != nil {
        if err != io.EOF {
            fmt.Println("read error:", err)
        }
        break
    }
```



Делаем запрос к HTTP серверу (GET)

```
nc localhost 5001
```

```
GET / HTTP/1.1
```

```
echo -e "GET / HTTP/1.1\n" | nc localhost 5001
```

```
00000000 47 45 54 20 2f 20 48 54 54 50 2f 31 2e 31 0a GET / HTTP/1.1
0000000F 0a .
00000000 48 54 54 50 2f 31 2e 31 20 32 30 30 20 4f 4b 0d HTTP/1.1 200 OK.
00000010 0a 53 65 72 76 65 72 3a 20 57 65 72 6b 7a 65 75 .Server: Werkzeug
00000020 67 2f 33 2e 30 2e 31 20 50 79 74 68 6f 6e 2f 33 g/3.0.1 Python/3
00000030 2e 31 31 2e 34 0d 0a 44 61 74 65 3a 20 4d 6f 6e .11.4..Date: Mon
00000040 2c 20 31 39 20 46 65 62 20 32 30 32 34 20 31 36 , 19 Feb 2024 16
00000050 3a 30 31 3a 35 33 20 47 4d 54 0d 0a 43 6f 6e 74 :01:53 GMT..Cont
00000060 65 6e 74 2d 54 79 70 65 3a 20 61 70 70 6c 69 63 ent-Type : applic
00000070 61 74 69 6f 6e 2f 6a 73 6f 6e 0d 0a 43 6f 6e 74 ation/js on..Cont
00000080 65 6e 74 2d 4c 65 6e 67 74 68 3a 20 39 33 0d 0a ent-Leng th: 93..
00000090 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 63 6c 6f 73 Connecti on: clos
000000A0 65 0d 0a 0d 0a e....
000000A5 7b 0a 20 20 22 63 6f 6f 6b 69 65 22 3a 20 7b 7d {, "cookie": {}
000000B5 2c 0a 20 20 22 67 65 74 44 61 74 61 22 3a 20 7b ,, "get Data": {
000000C5 7d 2c 0a 20 20 22 68 65 61 64 65 72 73 22 3a 20 },, "headers":
000000D5 7b 7d 2c 0a 20 20 22 70 6f 73 74 44 61 74 61 22 {},, "postData"
000000E5 3a 20 6e 75 6c 6c 2c 0a 20 20 22 75 72 6c 70 61 : null,, "urlpa
000000F5 72 61 6d 22 3a 20 6e 75 6c 6c 0a 7d 0a ram": null.}
```

CRLF символы

```
echo -en "GET / HTTP/1.1\r\n\r\n" | nc localhost 5001
```

```
echo -en "GET / HTTP/1.1\r\n" | nc localhost 5001
```

11	16:05:20,572190	127.0.0.1	127.0.0.1	TCP	68	60742 → 5001 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2902712752
12	16:05:20,572501	127.0.0.1	127.0.0.1	TCP	68	5001 → 60742 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2902712752
13	16:05:20,572539	127.0.0.1	127.0.0.1	TCP	56	60742 → 5001 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=2902712752 TSecr=2434937126
14	16:05:20,572568	127.0.0.1	127.0.0.1	TCP	72	60742 → 5001 [PSH, ACK] Seq=1 Ack=1 Win=408256 Len=16 TSval=2902712752 TSecr=2434937126
15	16:05:20,572577	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 5001 → 60742 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=2902712752 TSecr=2434937126
16	16:05:20,572581	127.0.0.1	127.0.0.1	TCP	56	60742 → 5001 [FIN, ACK] Seq=17 Ack=1 Win=408256 Len=0 TSval=2902712752 TSecr=2434937126
17	16:05:20,572609	127.0.0.1	127.0.0.1	TCP	56	5001 → 60742 [ACK] Seq=1 Ack=17 Win=408256 Len=0 TSval=2434937126 TSecr=2902712752
18	16:05:20,572625	127.0.0.1	127.0.0.1	TCP	56	5001 → 60742 [ACK] Seq=1 Ack=18 Win=408256 Len=0 TSval=2434937126 TSecr=2902712752
19	16:05:20,580365	127.0.0.1	127.0.0.1	TCP	221	5001 → 60742 [PSH, ACK] Seq=1 Ack=18 Win=408256 Len=165 TSval=2434937135 TSecr=2902712752
20	16:05:20,580422	127.0.0.1	127.0.0.1	TCP	56	60742 → 5001 [ACK] Seq=18 Ack=166 Win=408128 Len=0 TSval=2902712761 TSecr=2434937135
21	16:05:20,580451	127.0.0.1	127.0.0.1	HTTP/JSON	149	HTTP/1.1 200 OK , JSON (application/json)

http							
No.	Time	Source	Destination	Protocol	Length	Key	Info
21	16:05:20,580451	127.0.0.1	127.0.0.1	HTTP/JSON	149		HTTP/1.1 200 OK , JSON (application/json)

Подсчет Content-Length

Response	= Status-Line	; Section 6.1
	*((general-header	; Section 4.5
	response-header	; Section 6.2
	entity-header) CRLF)	; Section 7.1
	CRLF	
	[message-body]	; Section 7.2

- Считаем message-body размер (без учета CRLF протокола)

```
curl -F file=@06.txt http://localhost:5001 --trace request.txt
```

```
=> Send data, 205 bytes (0xcd)
0000: 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d -----
0010: 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 62 30 34 7a 38 6b -----b04z8k
0020: 38 32 6a 32 32 47 62 77 4b 5a 4a 72 31 70 4a 73 82j22GbWkZJr1pJs
0030: 0d 0a 43 6f 6e 74 65 6e 74 2d 44 69 73 70 6f 73 ..Content-Dispos
0040: 69 74 69 6f 6e 3a 20 66 6f 72 6d 2d 64 61 74 61 ition: form-data
0050: 3b 20 6e 61 6d 65 3d 22 66 69 6c 65 22 3b 20 66 ; name="file"; f
0060: 69 6c 65 6e 61 6d 65 3d 22 30 36 2e 74 78 74 22 ilename="06.txt"
0070: 0d 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 ..Content-Type:
0080: 74 65 78 74 2f 70 6c 61 69 6e 0d 0a 0d 0a 68 65 text/plain...he
0090: 6c 6c 6f 0a 31 32 33 0d 0a 2d 2d 2d 2d 2d 2d 2d llo.123..-----
00a0: 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d 2d -----
00b0: 2d 2d 2d 62 30 34 7a 38 6b 38 32 6a 32 32 47 62 ---b04z8k82j22Gb
00c0: 77 4b 5a 4a 72 31 70 4a 73 2d 2d 0d 0a          wkZJr1pJs---..
== Info: We are completely uploaded and fine
```

```
=> Send data, 15 bytes (0xf)
0000: 71 77 65 3d 48 65 6c 6c 6f 0a 57 6f 72 6c 64     qwe=Hello.World
```

```
curl --data $'qwe=Hello\nWorld' http://localhost:5001 --trace request.txt
```

Делаем запрос к HTTP серверу (POST)

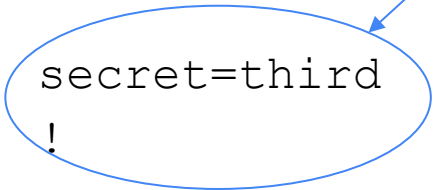
POST /flag5 HTTP/1.1

Accept: */*

Content-Length: 14

Content-Type: application/x-www-form-urlencoded

secret=third
!



```
{  
  "cookie": {},  
  "getData": {},  
  "headers": {  
    "Accept": "*/*",  
    "Content-Length": "14",  
    "Content-Type": "application/x-www-form-urlencoded"  
  },  
  "postData": {  
    "secret": "third\n!"  
  },  
  "urlparam": "flag5"  
}
```

Сервер через nc, клиент через curl

```
curl localhost:8000/index.html
#<1
→ ~ curl localhost:8000/index.html
#<2
→ ~ nc -l 8000
GET /index.html HTTP/1.1
Host: localhost:8000
User-Agent: curl/8.4.0
Accept: */*
[]
```

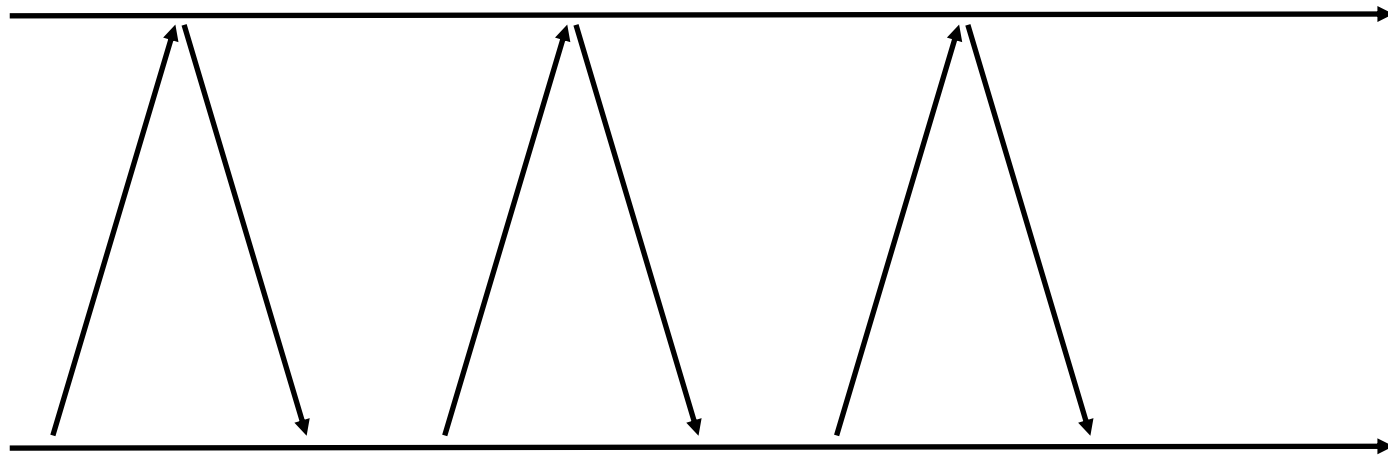
```
nc -l 8000
#<1
→ ~ curl localhost:8000/index.html
#<2
HTTP/1.1 200 Everything Is Just Fine
Server: netcat!
Content-Type: text/html; charset=UTF-8
<!doctype html>
<html>
<body>
<h1>A webpage served with netcat</h1>
</body>
</html>
[]
```

Отдаём страницу в цикле

```
nostr@MacBook-Pro-Aleksandr-5:~  
%$ 1  
→ ~ curl localhost:8000/index.html  
  
<!doctype html>  
<html><body><h1>A webpage served by  
  netcat</h1></body></html>  
→ ~ curl localhost:8000/index.html  
  
<!doctype html>  
<html><body><h1>A webpage served by  
  netcat</h1></body></html>  
→ ~   
  
%$ 2  
→ ~ cat index.http  
HTTP/1.1 200 OK  
Content-Type: text/html; charset=UTF-8  
Server: netcat!  
  
<!doctype html>  
<html><body><h1>A webpage served by  
  netcat</h1></body></html>  
→ ~ while true; do cat index.http  
  | nc -l 8000; done  
GET /index.html HTTP/1.1  
Host: localhost:8000  
User-Agent: curl/8.4.0  
Accept: */*  
  
GET /index.html HTTP/1.1  
Host: localhost:8000
```

Polling & Ajax

Short Polling



Медленно

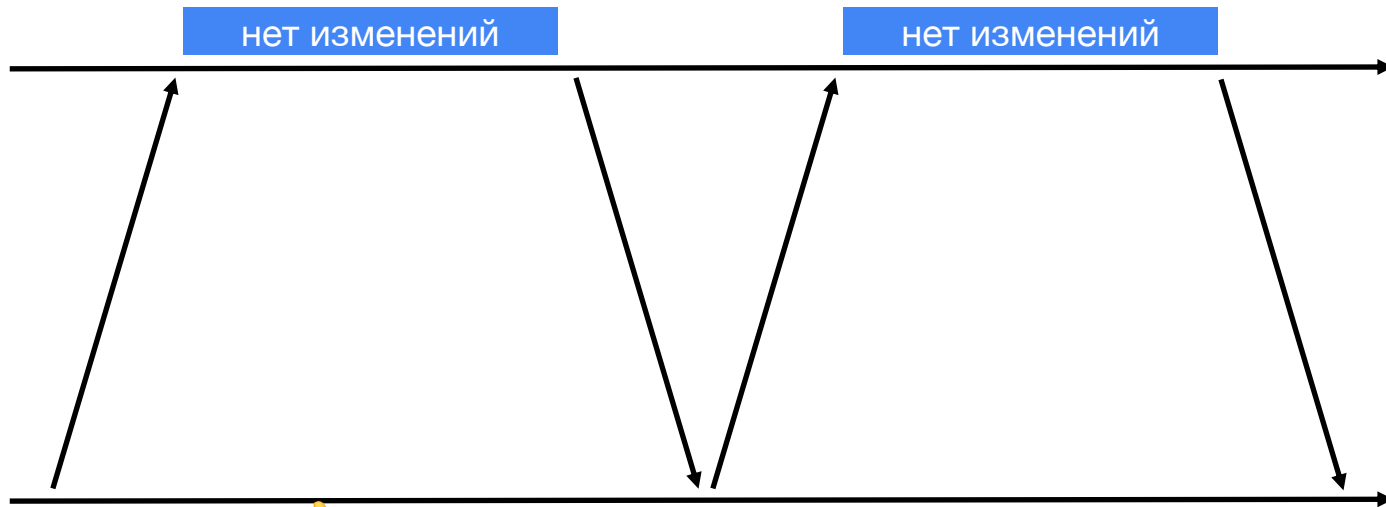


Требует ресурсов



Просто

Long Polling



Быстрые обновления



Чуть меньше ресурсов



Костыльно



Подвешивает 1 сокет

Websockets



Current aligned Usage relative Date relative Apply filters Show all ?									
IE	Edge	Firefox	Chrome	Safari	Opera	iOS Safari	Opera Mini	Android Browser	Opera Mobile
		2-3.6		3.1-4					
		¹ 4-5	¹ 4-14	¹ 5-5.1	10.1	3.2-4.1			
6-9		² 6-10	² 15	² 6-6.1	¹ 11.5	¹ 4.2-5.1		2.1-4.3	¹ 12
10	12-79	11-73	16-79	7-12.1	12.1-65	6-13.2		4.4-4.4.4	12.1
11	80	74	80	13	66	13.3	all	80	46
		75-76	81-83	13.1-TP		13.4			



Реалтайм



Двунаправленные



Эффективные



Git

Основные концепции Git, которые нужно знать

- Repository
- Staging
- Commit
- Push/Pull
- Branch
- Merge

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

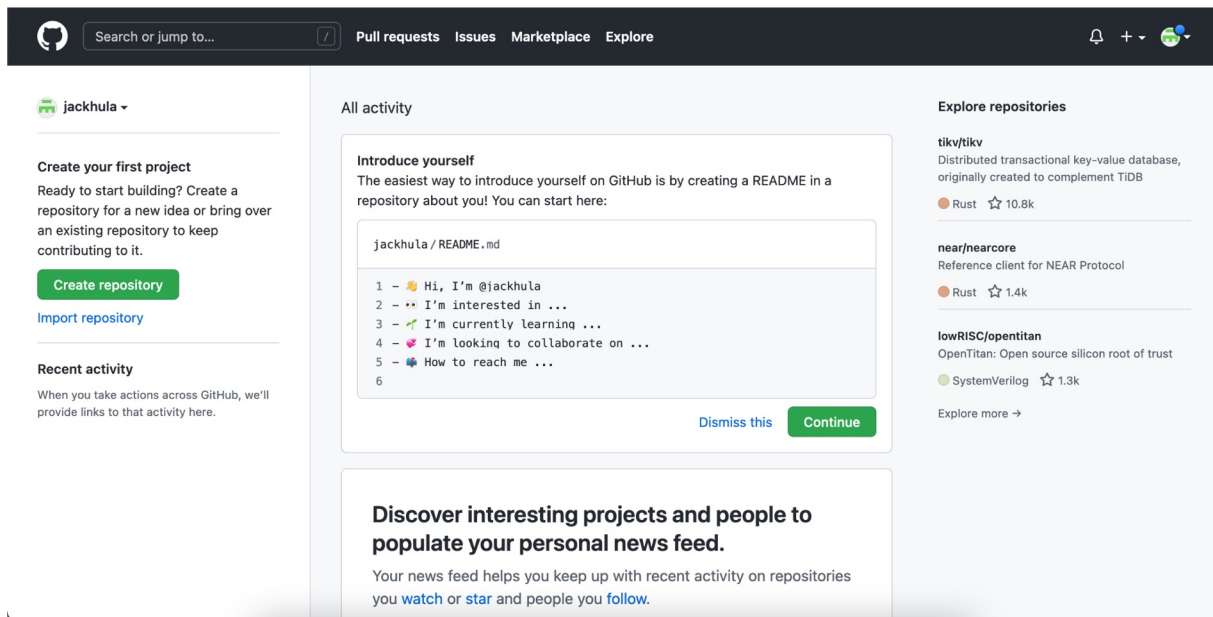
AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

<https://www.w3schools.com/git/default.asp?remote=github>

<https://www.w3schools.com/git/exercise.asp>



GitHub



The screenshot shows the GitHub homepage for a user named 'jackhula'. The top navigation bar includes the GitHub logo, a search bar, and links for 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. The main content area is divided into three columns. The left column, titled 'jackhula', contains a 'Create your first project' section with a 'Create repository' button and an 'Import repository' link, followed by a 'Recent activity' section. The middle column, titled 'All activity', features an 'Introduce yourself' section with a 'Continue' button and a 'Dismiss this' link, and a 'Discover interesting projects and people to populate your personal news feed.' section. The right column, titled 'Explore repositories', lists several repositories including 'tikv/tikv', 'near/nearcore', and 'lowRISC/opentitan'.

jackhula

Create your first project
Ready to start building? Create a repository for a new idea or bring over an existing repository to keep contributing to it.

[Create repository](#)

[Import repository](#)

Recent activity
When you take actions across GitHub, we'll provide links to that activity here.

All activity

Introduce yourself
The easiest way to introduce yourself on GitHub is by creating a README in a repository about you! You can start here:

```
jackhula / README.md
```

```
1 - 🙋 Hi, I'm @jackhula
2 - .. I'm interested in ...
3 - 🌱 I'm currently learning ...
4 - 💡 I'm looking to collaborate on ...
5 - 📫 How to reach me ...
6
```

[Dismiss this](#) [Continue](#)

Discover interesting projects and people to populate your personal news feed.
Your news feed helps you keep up with recent activity on repositories you [watch](#) or [star](#) and people you [follow](#).

Explore repositories

tikv/tikv
Distributed transactional key-value database, originally created to complement TiDB
Rust ⭐ 10.8k

near/nearcore
Reference client for NEAR Protocol
Rust ⭐ 1.4k

lowRISC/opentitan
OpenTitan: Open source silicon root of trust
SystemVerilog ⭐ 1.3k

[Explore more](#) →



VPN сеть на LMS2.0

Скачиваем персональный VPN конфиг

Курсы  Открытый ответ  Результаты

 VPN  

Информация о подключении к VPN

Конфигурационный файл:

[Скачать](#)

Инструкция (Windows):

[Скачать](#)

Инструкция (Linux):

[Скачать](#)

Инструкция (MacOS):

[Скачать](#)

Интерфейс: Polygon-student-61

Статус: ☐ Отключен

Публичный ключ: p9TrtTtyGDMpcoaovrlniGkNtPP4L25zMs/BfCdcewQ=

IP-адреса: 10.10.20.68/32

[Подключен](#)

Задания, для решения которых нужен VPN

Развертывание

Перезагрузить

Уничтожить

11:59:55

10.10.10.10:32776

```
→ ~ ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10): 56 data bytes
64 bytes from 10.10.10.10: icmp_seq=0 ttl=63 time=8.953 ms
64 bytes from 10.10.10.10: icmp_seq=1 ttl=63 time=10.023 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=63 time=19.816 ms
^C
--- 10.10.10.10 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 8.953/12.931/19.816/4.888 ms
→ ~ █
```

Полезные ссылки

- HTTP
 - <https://datatracker.ietf.org/doc/html/rfc4918#section-9.8>
 - <https://www.w3.org/Protocols/rfc2616/rfc2616-sec5.html>
 - [https://gchq.github.io/CyberChef/#recipe=URL_Encode\(false\)&input=VVJMIEVuY29kaW5n](https://gchq.github.io/CyberChef/#recipe=URL_Encode(false)&input=VVJMIEVuY29kaW5n)
 - <https://www.w3.org/TR/html401/interact/forms.html#h-17.13.4.2>
- Инструменты
 - <https://curl.se/>
 - <https://curl.se/windows/>
 - <https://www.postman.com/downloads/>
 - <https://eternallybored.org/misc/netcat/>

Вопросы?